

Northwestern Trading Contest

Powered by NUFintech

On behalf of Northwestern Fintech, welcome to the second annual Northwestern Trading Competition! The contest consists of two main algorithmic cases and one manual case. All algorithms will be written in the week leading up to the competition, and will be run in real-time at the event. The competition will run from 9am to 5pm on October 18th, breakfast and lunch will be provided

The contest would not be possible without the work of many members of NUFT, particularly the NUTC development team, as well as other trading contest veterans. Additionally, this event is made possible by the generous support of our competition sponsors and supporters - IMC Trading, HRT, Five Rings, Chicago Trading Company, and All Options as well as the Northwestern Associated Student Government, the Northwestern Murphy Society, and the McCormick School of Engineering.

Join the discord for announcements and questions not covered in this document: [link](#)

Guidelines

1. Participants may compete in teams of 2. If participating in a pair, participants **must** complete the "Group Settings" tab on the NUTC dashboard.
2. Participants must be in person for their algorithm to be run in the contest on October 18th.
3. We highly encourage participants to test their algorithms in the **sandbox** - more information below.
4. Any attempt to break the exchange or platform will result in **immediate disqualification**
 - a. This includes attempting to make network calls, interact with resources outside of the algorithm, or intentionally memory leak/overload the runtime(you won't be able to do this accidentally).
 - b. If you have questions, feel free to ask in the discord.

Schedule

Date, Time	Event
10/11, 2:30 PM	Development Starts
10/17, 11:59 PM	Submission Deadline
-	-

10/18, 9AM (Ford Design Center, The Hive)	Breakfast, Arrivals
10/18 10:00 AM	Contest Opening, HFT Case Overview
10/18 10:30 AM	HFT Case Run
10/18 11:30 AM	Manual Trading Game
10/18 12:10 PM	Lunch
10/18 12:50 PM	Sponsor Presentations, Merch, Networking
10/18, 2:30 PM	Crypto Case Run
10/18 3:30PM - 5:00 PM	Winners Announced, Networking, End of Competition

Awards

Each case will be scored according to the competitor(s) PnL (profit and loss). In other words, we score by the combined capital and portfolio value¹. Competitors must be present in order to receive prizing.

For each of the two algorithmic cases, first place will receive \$700, second will receive \$400, and third will receive \$150. First place in the manual trading game will receive \$300. We are still negotiating with sponsors and these amounts may increase.

Competition Technology

All cases are run on the NUTC proprietary exchange platform, built in-house by the NU Fintech exchange team in C++. Submissions are to be done in Python or C++ using the **templates provided on the contest dashboard (nutc.nuft.dev/dash)**. All submissions will be done through the platform, and are due by 11:59PM on 10/17. Submissions after the deadline will not be accepted. Algorithms that fail the submission process will not be eligible to participate. We urge competitors to use the provided sandbox to test the behavior of their algorithms.

The exchange contains many bots, including market makers, retail traders, and institutional traders. These create a market in the background that can and should be used to your advantage.

We have a small dev team covering a codebase of 30,000+ lines. Please let us know if you run into issues. Bug reports on discord are highly appreciated.

¹ Positions held at the end of the contest will be liquidated at market value.

Algorithmic Cases Instructions

You may write your algorithm in Python or C++. We recommend using C++ if your strategy relies on low-latency/short horizons (such as speed arbitrage), but for less time-sensitive strategies, Python works well.

You now have access to the [NUTC dashboard](#), which is used for submission, partner settings, FAQ, etc. Download the Python or C++ algorithm template from the dashboard. You should submit an algorithm for each case before the deadline.

The latest submitted algorithm for each case is the one we will run - regardless of whether it works.

Writing your Algorithm

The following instructions are written for Python, but are analogous to the C++ template. You may use C++ features up to C++20.

Callback Functions

The template has four callback functions that are called **by the exchange** to **provide you information**: `__init__`, `on_orderbook_update`, `on_trade_update`, and `on_account_update`.

`__init__` is run once when your algorithm starts and may be used for initialization.

`on_orderbook_update` is called when the quantity of a stock available at a given level has changed. It gives you the **total quantity** available at a **given price**. For example, `on_orderbook_update` with `price=3`, `quantity=10` means the orders at `price=3` sum to 10 quantity. This could be one order of `q=10`, two orders of `q=5`, etc.

`on_trade_update` indicates that two orders in the exchange matched. These may be anyone's orders, including yours. The *side* reflects the aggressive side.

`on_account_update` indicates that one of **your** orders matched with another, and includes your new `capital_remaining`.

Order Functions

Additionally, three functions are exposed to allow you to interact with the exchange: `place_market_order`, `place_limit_order`, and `cancel_order`.

Place_market_order places an order that will immediately match at the best price for the given ticker.

Place_limit_order places an order that will attempt to match at the given price. This function returns an **order_id**. If IOC(immediate-or-cancel) is set to true, it will be canceled if it doesn't match immediately. Otherwise, **the order must be canceled**. We have reasonable limits for the amount of open orders - just make sure you aren't forgetting to cancel orders.

Cancel_order is used for limit orders where IOC=false. Provide a ticker and order_id.

Submission/Sandbox

You should submit for both cases by the competition. When you submit, your algorithm will be linted for errors, then placed into the NUTC sandbox for five minutes. The sandbox runs the exchange, bots, and any other algorithms submitted in the last five minutes against each other, and displays your results in real time. You can download the Python submission as well as the debug logs after completion.

If your algorithm is syntactically incorrect or immediately produces an error, you will be notified on submission.

While sandbox results can verify your algorithm's behavior, they do not guarantee success in the actual competition, which is longer, has a higher tick rate, and includes other competitors, potentially affecting performance. However, if your algorithm consistently loses money, make sure you aren't just trading the spread ;)

The final algorithm uploaded to the platform before the deadline will be run at the contest.

Cases

HFT

This case simulates the role of a high frequency actor in a time of high volatility. Data can be assumed to be near instant, and trading fees are completely disabled, allowing almost any variety of strategy. This is a competitive market, meaning all participants are contributing to the same orderbook. Much effort has been invested in ensuring the performance and fairness of the exchange, even under extreme order flow. A more generous rate limit is applied in this case.

Crypto Trading

This case is similar to case 1, but trading fees are set to 40 bips (0.4%) of traded volume and volatility is somewhat lower. The case remains judged on PnL, and all participants are still on the same exchange.

Advice

1. Sandbox results often differ from competition results. Do not entirely rely on them
2. Ensure your algorithm is error-free. The easiest way to lose is to write an algorithm that triggers an error (say, looking up a key in a dictionary when the key does not exist).
3. The most successful strategies are often (but not always) relatively simple
4. We have reasonable CPU and RAM limits (half a core and 1GB respectively). The latter will kill your algorithm if you exceed it. If you have a really cool ML idea, let us know and we might increase it for you.
5. `Place_market_order`, `place_limit_order`, and `cancel_order` are rate limited. You cannot increase the number of orders submitted via multithreading or other means.
6. Do not write code outside of the `Strategy` class.
7. You start with 100,000 in capital and 0 in stock. You cannot leverage, so you must buy stock from bots or others. (*hint: `on_trade_update` tells you what's available for purchase*)
8. You may import any standard Python packages, as well as the following
 - a. Numpy, pandas, polars, scipy, sortedcontainers, scikit-learn
9. You may use all of `stdlib (c++20)` for C++ algorithms
10. Limit orders with `ioc=false` MUST be canceled. Use them wisely.

Manual Trading Game

Participants will be split into teams of 6. The rules for the manual trading game will be released on the day of the competition to encourage quick strategizing 🤔

Support

If you encounter a bug, question not answered in this document, or otherwise have questions/feedback, please feel free to reach out on [discord](#).